

Optimización de Políticas de Juego en Entornos Dinámicos (Tetris) mediante Aprendizaje por Refuerzo y Extracción de Características

Equipo del curso CC421 – Inteligencia Artificial

1 de julio de 2026

Resumen

El juego Tetris constituye un problema clásico de Inteligencia Artificial debido a la naturaleza secuencial de las decisiones, el elevado número de estados posibles y la incertidumbre introducida por la aparición aleatoria de nuevas piezas. Estas características convierten al problema en un entorno adecuado para el estudio y evaluación de técnicas de Aprendizaje por Refuerzo y métodos de optimización de políticas.

El presente trabajo propone el desarrollo de un agente inteligente capaz de aprender estrategias de juego para Tetris mediante una representación basada en características geométricas del tablero y una aproximación lineal de la función de valor. El entorno se modela formalmente como un Proceso de Decisión de Markov, mientras que la calidad de cada estado se estima a partir de un conjunto de atributos que incluyen la altura agregada, el número de huecos, la rugosidad de la superficie, las líneas eliminadas, la altura máxima y los pozos.

Con el propósito de optimizar los parámetros de la función de valor se estudiaron dos enfoques complementarios. El primero emplea Aprendizaje por Diferencias Temporales TD(0), permitiendo la actualización incremental de los pesos a partir de la experiencia del agente. El segundo utiliza el Método de Entropía Cruzada, el cual optimiza directamente los parámetros de la política mediante un procedimiento de búsqueda estocástica basado en poblaciones.

Para el agente TD(0) se realizó un barrido exhaustivo de veintisiete configuraciones de hiperparámetros, cuya mejor política elimina en promedio 409.13 líneas antes de alcanzar el estado terminal. Al comparar ambos métodos bajo un protocolo de evaluación común, el Método de Entropía Cruzada igualó o superó a TD(0) (371.4 frente a 338.6 líneas promedio, con menor variabilidad) y, a diferencia de este, mantuvo el vector de pesos acotado e interpretable, evitando la divergencia numérica característica del aprendizaje semi-gradiente. Los resultados experimentales demuestran que la combinación de modelado formal mediante MDP, representación basada en características y optimización de la función de valor constituye una estrategia efectiva para resolver problemas de toma de decisiones secuenciales en entornos dinámicos de gran complejidad.

Palabras clave: Aprendizaje por Refuerzo, Tetris, Proceso de Decisión de Markov, Temporal Difference Learning, Cross-Entropy Method, Aproximación de Funciones.

1. Introducción

La Inteligencia Artificial ha experimentado un crecimiento significativo en las últimas décadas, impulsando el desarrollo de sistemas capaces de aprender, razonar y tomar decisiones en entornos de elevada complejidad. Dentro de este contexto, el Aprendizaje por Refuerzo constituye una de

las áreas de investigación más relevantes, debido a su capacidad para construir agentes que mejoran su comportamiento a partir de la interacción directa con el entorno.

Los videojuegos representan un escenario particularmente adecuado para la evaluación de algoritmos de aprendizaje, ya que presentan problemas de decisión secuencial, objetivos claramente definidos y dinámicas complejas. Entre ellos, el juego Tetris ha sido ampliamente utilizado como un banco de pruebas para la investigación en Inteligencia Artificial debido a la gran dimensionalidad de su espacio de estados y al carácter estocástico de la secuencia de piezas.

Desde una perspectiva computacional, la obtención de una política óptima para Tetris resulta un problema extremadamente complejo. El número de configuraciones posibles del tablero crece de manera exponencial, haciendo inviable la utilización de métodos de búsqueda exhaustiva o técnicas de enumeración completa del espacio de estados.

Ante esta dificultad, se han propuesto diversas estrategias basadas en funciones heurísticas, métodos evolutivos y algoritmos de Aprendizaje por Refuerzo. Particularmente, las aproximaciones basadas en funciones de valor y representaciones compactas del tablero han demostrado ser capaces de producir políticas competitivas con un costo computacional relativamente bajo.

El presente trabajo aborda el problema de Tetris mediante la construcción de un agente inteligente basado en una aproximación lineal de la función de valor. El entorno se modela formalmente como un Proceso de Decisión de Markov y los estados se representan mediante un conjunto de características geométricas extraídas del tablero de juego.

Asimismo, se estudian dos estrategias de optimización de los parámetros de la función de valor. La primera emplea el algoritmo de Diferencias Temporales TD(0), permitiendo un aprendizaje incremental basado en la experiencia del agente. La segunda utiliza el Método de Entropía Cruzada, un procedimiento de optimización estocástica que busca directamente configuraciones de parámetros de alto rendimiento.

El objetivo principal del trabajo consiste en analizar la capacidad de estos métodos para aprender políticas de juego eficientes y estudiar la influencia de los hiperparámetros sobre el desempeño final del agente.

2. Estado del Arte

El problema de Tetris ha sido objeto de investigación durante más de tres décadas y constituye uno de los entornos clásicos para el estudio de algoritmos de Inteligencia Artificial y Aprendizaje por Refuerzo. Su interés radica en la elevada dimensionalidad del espacio de estados, la incertidumbre introducida por la generación aleatoria de piezas y la necesidad de tomar decisiones secuenciales de largo plazo (Russell and Norvig, 2021; Sutton and Barto, 2018).

Los primeros trabajos se basaron en agentes heurísticos capaces de evaluar la calidad de un tablero mediante un conjunto de características geométricas. Entre ellos destaca el controlador propuesto por Dellacherie (2003), el cual utiliza medidas como la altura de las columnas, el número de huecos y la irregularidad de la superficie para determinar la mejor acción posible.

Posteriormente, diversos investigadores exploraron técnicas de optimización de políticas. Szita y Lőrincz (2006) introdujeron el Método de Entropía Cruzada para el aprendizaje automático de agentes de Tetris, demostrando que los métodos de optimización estocástica pueden superar el rendimiento de numerosos controladores heurísticos.

Con el desarrollo del Aprendizaje por Refuerzo, surgieron nuevas aproximaciones basadas en la estimación de funciones de valor. Thiery y Scherrer (2009) propusieron controladores fundamentados en la evaluación de afterstates y en la aproximación de funciones, obteniendo políticas de alto desempeño mediante representaciones compactas del tablero.

Posteriormente, Gabillon, Ghavamzadeh y Scherrer (2013) demostraron que las técnicas de Programación Dinámica Aproximada pueden alcanzar desempeños competitivos en Tetris, evidenciando la importancia de la representación del estado y de las funciones de valor aproximadas.

Más recientemente, el auge del Aprendizaje Profundo ha permitido la construcción de agentes

basados en Deep Reinforcement Learning, utilizando redes neuronales profundas para aproximar funciones de valor y políticas de decisión. No obstante, estos métodos suelen requerir grandes volúmenes de datos, recursos computacionales considerables y tiempos de entrenamiento significativamente mayores (Goodfellow et al., 2016; Sutton and Barto, 2018).

En contraste, las aproximaciones lineales continúan siendo una alternativa atractiva debido a su bajo costo computacional, facilidad de interpretación y capacidad para proporcionar resultados competitivos en problemas de mediana complejidad (Thiery and Scherrer, 2009).

Por estas razones, el presente trabajo adopta un enfoque basado en representación mediante características y aproximación lineal de funciones, incorporando tanto Aprendizaje por Diferencias Temporales como el Método de Entropía Cruzada para el aprendizaje y optimización de las políticas de juego.

3. Modelo: formulación como MDP

Modelamos Tetris como un proceso de decisión de Markov (MDP) episódico $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$. El carácter *episódico* proviene de que toda partida termina en un estado absorbente (*top-out*) tras un número finito de pasos, lo que da un retorno bien definido sin necesidad de descuento.

3.1. Elementos

- **Estado** $s = (b, p)$: tablero $b \in \{0, 1\}^{H \times W}$ ($H = 20$, $W = 10$) y pieza actual $p \in \{1, \dots, 7\}$. Un estado alcanzable nunca contiene una fila completa, pues se elimina al fijar.
- **Acciones** $\mathcal{A}(s) = \{(r, c)\}$: una *colocación* es una rotación r y una columna de anclaje c factibles para la pieza actual (Sección 3.3).
- **Transición** $P(s' | s, a)$: factoriza en una parte determinista y una aleatoria (Sección 3.4).
- **Recompensa** $R(s, a) = L(s, a)$: líneas eliminadas por la colocación (Sección 3.5).
- **Descuento** $\gamma \in [0, 1]$; en este trabajo $\gamma = 1$ (Sección 3.6).
- **Retorno y objetivo**: el retorno es la suma descontada de recompensas a partir de t , $G_t = \sum_{k \geq 0} \gamma^k R_{t+k}$, y se busca maximizar el retorno esperado $\mathbb{E}[G_0]$.

3.2. Terminología

Un *tetrominó* es una pieza de cuatro celdas; empleamos los siete estándar (I, O, T, S, Z, J, L), cada uno con entre una y cuatro *rotaciones*. El *hard-drop* deja caer la pieza en línea recta hasta que apoya y la *fija* (*lock*); el *soft-drop* es una caída controlada que todavía permite maniobrar. Al fijarse, toda fila completa se elimina (*line clear*) y las superiores descienden; eliminar cuatro filas a la vez (el máximo posible, solo lograble con la pieza I en vertical) se denomina hacer un *Tetris*. El *afterstate* σ es el tablero resultante tras fijar y limpiar, antes de revelar la pieza siguiente (*spawn*). La partida termina en *top-out* cuando la nueva pieza no admite ninguna colocación. Llamamos *hueco* a una celda vacía con alguna celda ocupada por encima, *pozo* a una columna estrecha y profunda, y *altura* de la columna j a $h_j = H - \min\{i : b_{i,j} = 1\}$.

3.3. Acciones por colocación: finitud y cota

Cada acción es un par (r, c) . El número de rotaciones distintas de cualquier tetrominó es a lo sumo 4. Fijada una rotación de ancho w_r con $1 \leq w_r \leq 4$, el número de columnas de anclaje válidas es $W - w_r + 1$ (a lo sumo W). En consecuencia,

$$|\mathcal{A}(s)| \leq \sum_r (W - w_r + 1) \leq 4W,$$

y $\mathcal{A}(s)$ es finito por ser un producto de conjuntos finitos (rotaciones y columnas). El conteo real es menor, ya que se descartan las colocaciones que desbordan la pila. La formulación por colocación reduce el control a *una decisión por pieza*.

3.4. Transición y mapa de colocación

El estado siguiente tiene dos componentes, $s' = (\sigma', p')$ (el tablero y la pieza siguiente), así que la transición es una distribución *conjunta*. La descomponemos con la regla de la cadena:

$$P((\sigma', p') \mid s, a) = \mathbb{P}(\sigma' \mid s, a) \cdot \mathbb{P}(p' \mid \sigma', s, a). \quad (1)$$

El *mapa de colocación* $\sigma = f(s, a)$ es determinista: dado (s, a) , el hard-drop más la limpieza de líneas producen un *único* tablero. Una distribución con toda su masa en un solo valor es una delta, que escribimos con la función indicadora

$$\mathbf{1}\{C\} = \begin{cases} 1 & \text{si la condición } C \text{ es verdadera,} \\ 0 & \text{en caso contrario,} \end{cases} \quad (2)$$

de modo que el primer factor de (1) es

$$\mathbb{P}(\sigma' \mid s, a) = \mathbf{1}\{\sigma' = f(s, a)\}. \quad (3)$$

El segundo factor es la pieza siguiente, que se sortea de forma independiente del tablero, de la acción y de la historia (es exógena), y la modelamos uniforme sobre las siete:

$$\mathbb{P}(p' \mid \sigma', s, a) = \mathbb{P}(p') = \frac{1}{7}, \quad p' \sim \text{Unif}\{1, \dots, 7\}. \quad (4)$$

Sustituyendo (3) y (4) en (1) se obtiene la transición:

$$P((\sigma', p') \mid s, a) = \mathbf{1}\{\sigma' = f(s, a)\} \cdot \frac{1}{7}. \quad (5)$$

Es decir, desde (s, a) solo son alcanzables los siete sucesores $(\sigma, 1), \dots, (\sigma, 7)$ y equiprobables; para cualquier $\sigma' \neq f(s, a)$ la probabilidad es nula. Elegir el generador independiente (i.i.d.) preserva la propiedad de Markov sin ampliar el estado: un aleatorizador por bolsas (*7-bag*) obligaría a recordar las piezas ya emitidas. Esta factorización es la que habilita trabajar con el afterstate σ (Sección 5).

3.5. Recompensa y su rango

$L(s, a)$ es el número de filas completas eliminadas por la colocación. Como un tetrominó ocupa cuatro celdas repartidas en a lo sumo cuatro filas, y una fila solo puede completarse si la pieza aporta alguna celda a ella, se eliminan a lo sumo cuatro filas. El mínimo 0 (ninguna fila completada) y el máximo 4 (una *I* vertical que cierra cuatro filas a la vez, un *Tetris*) son alcanzables, de modo que

$$L(s, a) \in \{0, 1, 2, 3, 4\}.$$

3.6. Decisiones de modelado

Recompensa. Tomamos $R = L$ porque el objetivo del problema es maximizar el total de líneas de la partida: es una señal acotada y directamente alineada con la métrica de evaluación.

Descuento γ . El *objetivo* del problema es maximizar el total de líneas de la partida, por lo que en el retorno se fija $\gamma = 1$: toda línea vale igual sin importar cuándo se elimina. Al ser la tarea episódica y terminar en un número finito de pasos, el retorno no descontado G_t es una suma finita y está bien definida; en la práctica se acota la longitud del episodio para mantenerla acotada. Esto no impide que un *método de entrenamiento* emplee internamente un descuento $\gamma < 1$ como recurso algorítmico (por ejemplo, para estabilizar el *bootstrapping* en TD, Sección 6); dicho descuento interno no modifica el objetivo de desempeño, que se reporta siempre como total de líneas. El método que optimiza directamente este objetivo, sin descuento ni gradientes, se detalla en la Sección 7.

Abstracción del tiempo. El modelo prescinde de la tasa de refresco (*frame rate*) y del tiempo real: un paso del MDP equivale a *una pieza colocada*, y la dinámica intra-pieza (gravedad, soft-drop, desplazamientos laterales) se colapsa en la elección de la posición final más el hard-drop. Se asume, pues, una *decisión inmediata*: lo relevante para la calidad de la política es *dónde* reposa la pieza, no la trayectoria ni el cronometraje para llevarla allí. Como implicación directa, **no se modelan las maniobras que requieren rotar o desplazar la pieza durante la caída** (*tucks*, *T-spins*): $\mathcal{A}(s)$ enumera únicamente las caídas rectas alcanzables por hard-drop. Tampoco se modelan la aceleración de la gravedad por nivel ni el puntaje por soft-drop. El bucle de frames solo sería relevante para una demostración en vivo, donde la colocación elegida se traduciría a una secuencia de entradas de control.

4. Representación mediante características

El espacio de estados de Tetris es muy grande. Por ello, no resulta práctico almacenar un valor independiente para cada configuración posible del tablero. En su lugar, se utiliza una representación compacta basada en características numéricas del tablero, evaluadas sobre el *afterstate* $\sigma = f(s, a)$ resultante de colocar una pieza (Sección 3).

Sea el vector de características, con un orden fijo compartido con el vector de pesos w y con la implementación (`features.py`):

$$\varphi(\sigma) = (f_1(\sigma), f_2(\sigma), f_3(\sigma), f_4(\sigma), f_5(\sigma), f_6(\sigma)). \quad (6)$$

Cada componente resume una propiedad relevante del afterstate. Sea $b_{i,j} \in \{0, 1\}$ la ocupación de la celda en la fila i (numeradas desde arriba) y la columna j , con H filas y W columnas. La altura h_j de la columna j es la *posición de la celda ocupada más alta*:

$$h_j = H - \min\{i : b_{i,j} = 1\} \quad (h_j = 0 \text{ si la columna está vacía}). \quad (7)$$

4.1. Altura agregada

La altura agregada se define como la suma de las alturas de todas las columnas:

$$f_1(\sigma) = \sum_{j=1}^W h_j. \quad (8)$$

Un valor alto de esta característica indica que la pila de piezas se encuentra cerca de la parte superior del tablero.

4.2. Huecos

Un hueco es una celda vacía que tiene al menos una celda ocupada por encima en la misma columna. La cantidad total de huecos se representa como:

$$f_2(\sigma) = \sum_{j=1}^W |\{i : b_{i,j} = 0 \wedge \exists i' < i, b_{i',j} = 1\}|. \quad (9)$$

Esta característica es importante porque los huecos dificultan la eliminación de líneas y suelen empeorar la calidad del tablero.

4.3. Rugosidad

La rugosidad mide la irregularidad de la superficie del tablero. Se calcula como la suma de las diferencias absolutas entre alturas de columnas consecutivas:

$$f_3(\sigma) = \sum_{j=1}^{W-1} |h_j - h_{j+1}|. \quad (10)$$

Un tablero con alta rugosidad dificulta la colocación eficiente de nuevas piezas.

4.4. Líneas eliminadas

El número de líneas eliminadas por la colocación que produce el afterstate σ , es decir, $L(s, a)$ según la Sección 3:

$$f_4(\sigma) = L(s, a). \quad (11)$$

Esta característica se relaciona directamente con el objetivo principal del juego.

4.5. Altura máxima

La altura máxima corresponde a la columna más alta del tablero:

$$f_5(\sigma) = \max_{1 \leq j \leq W} h_j. \quad (12)$$

Esta característica permite estimar qué tan cerca se encuentra el agente del estado terminal.

4.6. Pozos

Un pozo es una columna o región vertical vacía rodeada por columnas de mayor altura. Su contribución es la suma de las profundidades de los pozos respecto a sus columnas vecinas, tratando las paredes laterales como vecinos de altura infinita:

$$f_6(\sigma) = \sum_{j=1}^W \max(0, \min(h_{j-1}, h_{j+1}) - h_j), \quad h_0 = h_{W+1} = +\infty. \quad (13)$$

Los pozos pueden ser útiles si permiten colocar una pieza larga, pero en general aumentan el riesgo si no son controlados adecuadamente.

En conjunto, las seis características ofrecen una representación compacta, en el orden fijo ya indicado (compartido con el vector de pesos w y con `features.py`), que reduce la complejidad del problema y habilita el uso de una función de valor lineal.

5. Valor de afterstate, aproximación lineal y política

5.1. Funciones de valor

Para una política π , la función de valor de estado es el retorno esperado (con el retorno G_t definido en la Sección 3) al partir de s y seguir π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] = \mathbb{E}_\pi\left[\sum_{k \geq 0} \gamma^k R_{t+k} \mid s_t = s\right], \quad (14)$$

con $s = (b, p)$. El valor de afterstate promedia V^π sobre la pieza siguiente, es decir, valora el tablero post-decisión σ antes de revelar p' :

$$U^\pi(\sigma) = \mathbb{E}_{p'}[V^\pi(\sigma, p')] = \sum_{p'} \mathbb{P}(p') V^\pi(\sigma, p'). \quad (15)$$

Distinguimos el valor bajo una política dada, U^π , del valor óptimo $U^* = U^{\pi^*}$; la ecuación de optimalidad de Bellman se expresa con U^* .

5.2. Ecuación de optimalidad de Bellman en forma de afterstate

El valor óptimo de estado satisface la ecuación de optimalidad de Bellman (Bellman, 1957; Sutton and Barto, 2018):

$$V^*(s) = \max_{a \in \mathcal{A}(s)} [R(s, a) + \gamma \mathbb{E}_{s'|s, a}[V^*(s')]]. \quad (16)$$

Sustituyendo la transición factorizada (Sección 3.4), con $s' = (\sigma, p')$ y $\sigma = f(s, a)$ determinista, la esperanza colapsa en el valor de afterstate:

$$\mathbb{E}_{s'|s, a}[V^*(s')] = \mathbb{E}_{p'}[V^*(f(s, a), p')] = U^*(f(s, a)), \quad (17)$$

de donde

$$V^*(s) = \max_{a \in \mathcal{A}(s)} [R(s, a) + \gamma U^*(f(s, a))]. \quad (18)$$

La ecuación (18) vale para *cualquier* estado. En particular, al evaluarla en el estado $s = (\sigma, p')$, cuyo tablero es el afterstate σ y cuya pieza actual es p' , se obtiene su valor óptimo:

$$V^*(\sigma, p') = \max_{a \in \mathcal{A}(\sigma, p')} [R((\sigma, p'), a) + \gamma U^*(f((\sigma, p'), a))]. \quad (19)$$

Tomando la esperanza sobre la pieza siguiente p' en ambos lados de (19) y usando la definición del valor de afterstate, $U^*(\sigma) = \mathbb{E}_{p'}[V^*(\sigma, p')]$ (ecuación (15) para la política óptima), el miembro izquierdo se convierte en $U^*(\sigma)$ y resulta la ecuación de optimalidad en forma de afterstate:

$$U^*(\sigma) = \mathbb{E}_{p'}\left[\max_{a \in \mathcal{A}(\sigma, p')} (R((\sigma, p'), a) + \gamma U^*(f((\sigma, p'), a)))\right]. \quad (20)$$

Esta ecuación caracteriza el valor óptimo de afterstate U^* , cuyo cálculo exacto es intratable por el enorme número de estados. Por ello, en lo que sigue se lo aproxima con una función lineal de las características (ecuación (23)), cuyos pesos se aprenden por TD(0) o por el Método de Entropía Cruzada, y de la que se deriva la política greedy (24).

5.3. Por qué se usa U^* en lugar de expandir la esperanza sobre p'

La acción greedy óptima admite dos formas equivalentes:

$$\pi^*(s) = \arg \max_a [R(s, a) + \gamma \mathbb{E}_{p'}[V^*(f(s, a), p')]], \quad (21)$$

$$\pi^*(s) = \arg \max_a [R(s, a) + \gamma U^*(f(s, a))]. \quad (22)$$

El paso de (21) a (22) es *exacto*: basta aplicar la definición del valor de afterstate, $U^*(\sigma) = \mathbb{E}_{p'}[V^*(\sigma, p')]$, con $\sigma = f(s, a)$.

Razón conceptual. La esperanza sobre p' describe lo que ocurre *después* de colocar la pieza actual y depende únicamente del tablero resultante σ , no de la acción a que se esté comparando (más allá de que a produce σ). Es, por tanto, una propiedad del afterstate y no de la decisión actual, por lo que puede extraerse del $\arg \max$ y quedar contenida una sola vez en $U^*(\sigma)$.

Sin modelo de la pieza siguiente al decidir. Al ser $U^*(\sigma)$ una función solo del tablero-afterstate, se la aproxima directamente con las características, $U^*(\sigma) \approx w^\top \varphi(\sigma)$. En tiempo de decisión, el agente únicamente enumera los afterstates $f(s, a)$, evalúa $w^\top \varphi$ sobre cada uno y elige el máximo; *nunca* necesita la distribución de p' , pues ese promedio quedó absorbido en U^* (en los pesos w) durante el aprendizaje. La forma (21), en cambio, exigiría al decidir conocer dicha distribución y evaluar V^* para las siete piezas de cada acción.

Eficiencia. Expandir la esperanza obligaría a promediar V^* sobre las siete piezas para cada una de las $|\mathcal{A}(s)|$ acciones candidatas, del orden de $|\mathcal{A}(s)| \cdot 7$ evaluaciones por decisión. Con (22), cada afterstate se puntúa con una sola evaluación $w^\top \varphi(\sigma)$, del orden de $|\mathcal{A}(s)|$; el factor siete se ahorra precisamente porque la aproximación de U^* evita recomputar el promedio en cada decisión.

Esta es la ventaja clásica de los estados post-decisión o *afterstates* (Sutton and Barto, 2018; Bertsekas and Tsitsiklis, 1996): al valorar el estado posterior a la jugada pero anterior al evento aleatorio, se separa la parte determinista (la elección del agente) de la estocástica (la pieza siguiente), y basta aprender el valor del resultado determinista.

5.4. Aproximación lineal y política greedy

Como U^* es desconocida, la aproximamos con una arquitectura lineal sobre el vector de características $\varphi(\sigma) \in \mathbb{R}^d$, definido en la Sección 4 (Bertsekas and Tsitsiklis, 1996):

$$\hat{U}(\sigma; w) = w^\top \varphi(\sigma), \quad w \in \mathbb{R}^d. \quad (23)$$

El vector de pesos w es el único parámetro que se aprende; lo determina un método de entrenamiento, ya sea TD(0) (Sección 6) o el Cross-Entropy Method (Sección 7). Sustituyendo (23) en la acción greedy (22) (con $\gamma = 1$) se obtiene la política, parametrizada por w :

$$\pi_w(s) = \arg \max_{a \in \mathcal{A}(s)} [R(s, a) + w^\top \varphi(f(s, a))]. \quad (24)$$

Es un look-ahead exacto de un paso: como $\mathcal{A}(s)$ es finito y f es computable, se enumeran todos los afterstates y se evalúa φ en cada uno. Dado que φ incluye la componente de líneas eliminadas, R queda absorbida y $\pi_w(s) = \arg \max_a w^\top \varphi(f(s, a))$. La política queda completamente determinada por w .

5.5. Objetivo de desempeño

Fijado w , queda fijada π_w ; su calidad es el retorno esperado por episodio, equivalentemente el valor esperado del estado inicial:

$$J(w) = \mathbb{E}_{\tau \sim \pi_w} [G_0] \stackrel{\gamma=1}{=} \mathbb{E}_{\tau \sim \pi_w} \left[\sum_{t=0}^T R_t \right] = \mathbb{E}_{s_0} [V^{\pi_w}(s_0)]. \quad (25)$$

Aquí $\tau = (s_0, a_0, R_1, s_1, a_1, \dots, s_T)$ es una *trayectoria* completa (una partida desde el inicio hasta el top-out), y la notación $\tau \sim \pi_w$ indica que esa trayectoria se *genera siguiendo* la política π_w : en cada paso la acción es $a_t = \pi_w(s_t)$ y la pieza siguiente se sortea $p' \sim \text{Unif}\{1, \dots, 7\}$. Como π_w es determinista, la única fuente de aleatoriedad de τ es la secuencia de piezas; por eso $J(w)$ promedia el total de líneas sobre todas las secuencias posibles. El método de la sección siguiente maximiza $J(w)$. Tres propiedades condicionan esa optimización:

- (I) **Estocástica:** la trayectoria τ es aleatoria (por la secuencia de piezas), así que $\sum_t R_t$ es una variable aleatoria y $J(w)$ solo se estima por Monte Carlo, promediando episodios.
- (II) **No diferenciable y constante a trozos:** π_w depende de w solo a través de $\arg \max_a w^\top \varphi$; un cambio pequeño de w no altera el máximo, y J solo salta cuando w cruza una frontera donde el $\arg \max$ cambia de colocación. Su gradiente es nulo casi en todas partes e indefinido en los saltos.
- (III) **Invariante a escala:** $J(\alpha w) = J(w)$ para todo $\alpha > 0$, pues escalar w multiplica todos los puntajes $w^\top \varphi$ por α sin cambiar el $\arg \max$; la política depende solo de la dirección de w . Por ello fijamos $\|w\|_2 = 1$.

Por (i)–(ii) los métodos de gradiente son inadecuados, lo que justifica el optimizador sin derivadas de la Sección 7.

6. Aprendizaje por Diferencia Temporal

El primer método de entrenamiento aprende la función de valor por *bootstrapping*, sin esperar al final del episodio, sobre la misma arquitectura lineal $\hat{U}(\sigma; w) = w^\top \varphi(\sigma)$ de la Sección 5. Es el baseline funcional del proyecto.

6.1. Error TD y actualización semi-gradiente

Para una transición entre afterstates $\sigma_t \rightarrow \sigma_{t+1}$ con recompensa R_{t+1} , el objetivo TD(0) y el error TD son

$$y_t = R_{t+1} + \gamma \hat{U}(\sigma_{t+1}; w), \quad (26)$$

$$\delta_t = y_t - \hat{U}(\sigma_t; w) = R_{t+1} + \gamma w^\top \varphi(\sigma_{t+1}) - w^\top \varphi(\sigma_t). \quad (27)$$

La actualización *semi-gradiente* desciende sobre $\frac{1}{2}\delta_t^2$ tratando el objetivo y_t como constante (no se deriva respecto a w):

$$w \leftarrow w + \alpha \delta_t \nabla_w \hat{U}(\sigma_t; w) = w + \alpha \delta_t \varphi(\sigma_t), \quad (28)$$

con tasa de aprendizaje α . En el estado terminal (top-out) se toma $\hat{U} = 0$, de modo que $\delta_T = R_T - w^\top \varphi(\sigma_T)$.

6.2. Política de comportamiento

Durante el entrenamiento se sigue una política ε -greedy sobre la acción greedy de (24): con probabilidad $1 - \varepsilon$ se elige $\arg \max_a w^\top \varphi(f(s, a))$ y con probabilidad ε una colocación al azar (exploración). Las tasas α y ε decaen con los episodios. El descuento $\gamma < 1$ (en la práctica $\gamma \in \{0,99, 0,999\}$, siendo 0,999 el de la mejor configuración) actúa como recurso algorítmico que estabiliza el *bootstrapping*; el desempeño que se reporta sigue siendo el total de líneas por partida.

6.3. Algoritmo

```

Inicializar  $w = 0$ 
para cada episodio:
     $s = \text{reset}()$ ;  $a = \text{epsilon\_greedy}(\text{legal\_placements}(s))$ 
    mientras no termine:
         $\text{sigma} = \text{afterstate}(s, a)$ 
         $(s', R, \text{fin}) = \text{step}(a)$ 
         $a' = \text{epsilon\_greedy}(\text{legal\_placements}(s'))$            # None si no hay jugada

```

```

sigma' = afterstate(s', a')                # terminal si a' = None
delta = R + gamma * U(sigma') - U(sigma)   # U(terminal) = 0
w = w + alpha * delta * phi(sigma)
s, a = s', a'

```

El entrenamiento se paraleliza de forma asíncrona: varios trabajadores juegan episodios y acumulan sus actualizaciones sobre un vector de pesos en memoria compartida (esquema tipo Hogwild!).

6.4. Estabilidad

La actualización (28) es *semi-gradiente*: no es el gradiente de una función objetivo fija, por lo que la combinación de *bootstrapping*, aproximación de funciones y muestreo puede hacer **divergir** los pesos (Sutton and Barto, 2018). En la práctica observamos que $\|w\|$ crece sin acotarse, aunque la política greedy sigue rindiendo gracias a la invarianza de escala (Sección 5). Esta limitación motiva el método sin *bootstrap* ni problema de escala de la Sección 7.

7. Optimización: Cross-Entropy Method

Optimizamos $\max_w J(w)$ con una familia de muestreo Gaussiana $g(w; \theta) = \mathcal{N}(w; \mu, \Sigma)$, con $\theta = (\mu, \Sigma)$.

7.1. Densidad objetivo y proyección por entropía cruzada

Para un nivel de desempeño λ , la densidad ideal concentra masa en la región élite:

$$q_\lambda(w) \propto \mathbf{1}\{J(w) \geq \lambda\} g(w; \theta).$$

Buscamos θ que minimice la divergencia de Kullback–Leibler (entropía cruzada) hacia q_λ :

$$\theta^* = \arg \min_{\theta} D_{\text{KL}}(q_\lambda \| g(\cdot; \theta)) = \arg \max_{\theta} \mathbb{E}_{q_\lambda} [\log g(w; \theta)].$$

Con muestras $w_1, \dots, w_N \sim g(\cdot; \theta_t)$ y nivel adaptativo λ_t igual al cuantil $(1 - \rho)$ de $\{J(w_i)\}$, el conjunto élite es $\mathcal{E}_t = \{w_i : J(w_i) \geq \lambda_t\}$, con $|\mathcal{E}_t| = \lceil \rho N \rceil$.

7.2. Solución en forma cerrada

El paso de maximización es un ajuste Gaussiano por máxima verosimilitud sobre los elites. Partiendo de $\log \mathcal{N}(w; \mu, \Sigma) = -\frac{1}{2} \log |2\pi\Sigma| - \frac{1}{2}(w - \mu)^\top \Sigma^{-1}(w - \mu)$ y anulando las derivadas,

$$\begin{aligned} \frac{\partial}{\partial \mu} \sum_{w \in \mathcal{E}} \log \mathcal{N} &= \Sigma^{-1} \sum_{w \in \mathcal{E}} (w - \mu) = 0 & \Rightarrow \quad \mu^* &= \frac{1}{|\mathcal{E}|} \sum_{w \in \mathcal{E}} w, \\ \frac{\partial}{\partial \Sigma^{-1}} \sum_{w \in \mathcal{E}} \log \mathcal{N} &= \frac{|\mathcal{E}|}{2} \Sigma - \frac{1}{2} \sum_{w \in \mathcal{E}} (w - \mu^*)(w - \mu^*)^\top = 0 & \Rightarrow \quad \Sigma^* &= \frac{1}{|\mathcal{E}|} \sum_{w \in \mathcal{E}} (w - \mu^*)(w - \mu^*)^\top. \end{aligned} \quad (29)$$

$$(30)$$

La actualización que minimiza la entropía cruzada es, pues, la media y la covarianza empíricas del conjunto élite. En la práctica se usa $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$ por eficiencia.

7.3. Inyección de ruido y evaluación Monte Carlo

El CEM puro hace $\Sigma_t \rightarrow 0$ y colapsa a un óptimo local; para mantener exploración se añade ruido decreciente:

$$\Sigma_{t+1} \leftarrow \Sigma^* + Z_t I, \quad Z_t = \max(0, a - t/b).$$

Cada $J(w_i)$ se estima por Monte Carlo sobre m episodios, $\hat{J}(w_i) = \frac{1}{m} \sum_{k=1}^m G^{(k)}(w_i)$, con $\text{Var}[\hat{J}] \propto 1/m$ (Rubinstein, 1999; Szita and Lőrincz, 2006).

7.4. Implementación

En este trabajo se emplea covarianza diagonal y el ruido decrece de forma lineal, $Z_t = c(1 - t/T)$, con T el número de generaciones. La evaluación de cada candidato usa un *tope de colocaciones* por partida (una buena política juega partidas casi ilimitadas) y *números aleatorios comunes*: todos los candidatos de una misma generación se evalúan con las mismas semillas de partida, lo que reduce la varianza de la comparación. El vector final se normaliza a $\|w\| = 1$; al ser la política invariante a escala, esto no altera el desempeño pero mantiene los pesos acotados, en contraste con la divergencia de magnitud observada en TD (Sección 6).

8. Algoritmo General de Entrenamiento

El procedimiento general de entrenamiento del agente se resume en el Algoritmo 1.

Algorithm 1 Entrenamiento del Agente

```
1: Inicializar el entorno Tetris
2: Inicializar los parámetros del modelo
3: for cada episodio de entrenamiento do
4:   Reiniciar el tablero
5:   Obtener el estado inicial
6:   while el episodio no haya terminado do
7:     Generar las acciones factibles
8:     Evaluar los afterstates
9:     Seleccionar una acción
10:    Ejecutar la acción
11:    Obtener recompensa y siguiente estado
12:    if se utiliza TD(0) then
13:      Actualizar los pesos mediante la ecuación de Diferencia Temporal
14:    end if
15:  end while
16: end for
17: if se utiliza CEM then
18:   Generar población de candidatos
19:   Evaluar cada individuo
20:   Seleccionar conjunto elite
21:   Actualizar la distribución de parámetros
22: end if
23: Devolver el mejor vector de pesos encontrado.
```

9. Arquitectura del Sistema

La arquitectura del sistema se organiza en módulos independientes que separan el entorno de simulación, la generación de acciones, la extracción de características, la evaluación de estados y los métodos de aprendizaje. Esta separación permite comparar TD(0) y el Método de Entropía Cruzada sobre una misma función de valor.

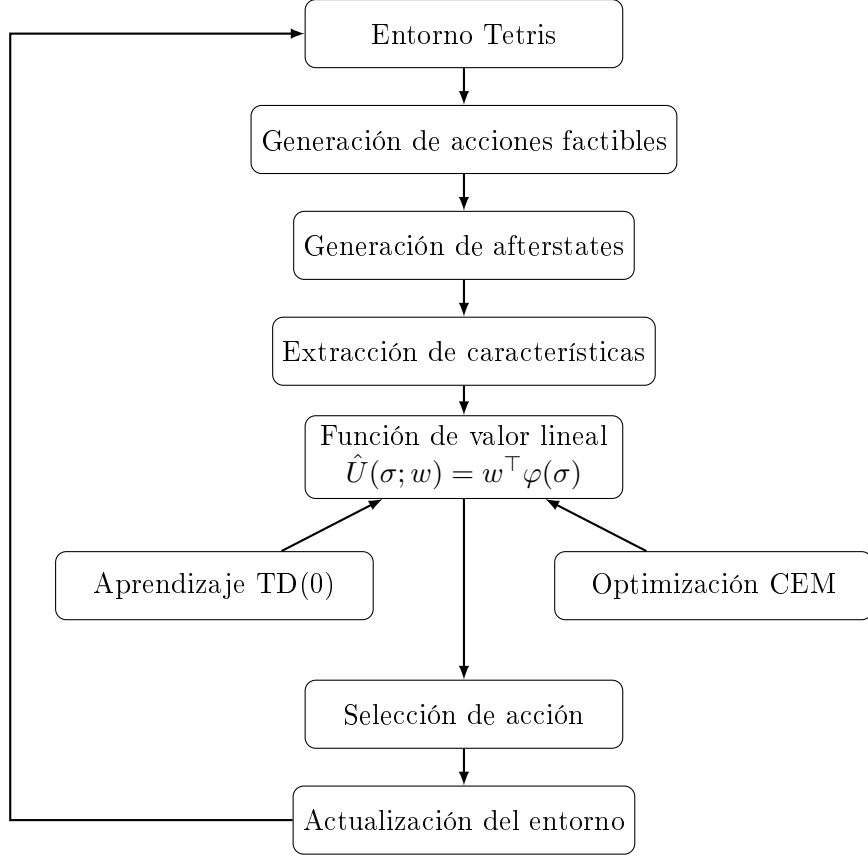


Figura 1: Arquitectura general del agente inteligente para Tetris.

El flujo inicia con el entorno de Tetris, que proporciona el tablero actual y la pieza activa. A partir de este estado se generan las acciones factibles, definidas por una rotación y una columna de colocación. Cada acción produce un tablero resultante o *afterstate*, que luego se transforma en un vector de características.

Los métodos TD(0) y CEM buscan obtener un vector de pesos adecuado para la función de valor. TD(0) actualiza los pesos de forma incremental durante la interacción con el entorno, mientras que CEM realiza una búsqueda poblacional sobre posibles vectores de pesos.

10. Diseño Experimental

Con el propósito de determinar la configuración de entrenamiento más adecuada para el agente de Aprendizaje por Refuerzo, se realizó un barrido exhaustivo de hiperparámetros mediante una estrategia de búsqueda en malla (*Grid Search*).

Se consideraron tres valores para el factor de descuento, tres valores para la tasa de aprendizaje y tres configuraciones para el decaimiento del parámetro de exploración, dando lugar a un total de

$$N_{\text{exp}} = 3 \times 3 \times 3 = 27 \quad (31)$$

configuraciones independientes de entrenamiento.

Los hiperparámetros evaluados fueron los siguientes:

- Factor de descuento:

$$\gamma \in \{0,90, 0,99, 0,999\}. \quad (32)$$

- Tasa de aprendizaje:

$$\alpha \in \{10^{-3}, 5 \times 10^{-4}, 10^{-4}\} \quad (33)$$

- Número de episodios de decaimiento de la exploración:

$$N_\epsilon \in \{1000; 1500; 1800\} \quad (34)$$

Conviene precisar el rol del factor de descuento en el barrido. El γ que se varía es el descuento *interno* del método TD(0) (Sección 6), empleado como recurso algorítmico para estabilizar el *bootstrapping*, y no el descuento del objetivo de desempeño, que se mantiene sin descontar ($\gamma = 1$, toda línea vale igual sin importar cuándo se elimina; Sección 3.6). Incluir γ en la malla no contradice esa elección: su propósito es *corroborar empíricamente*, en lugar de asumir, que un descuento interno cercano a 1 (el más fiel al objetivo) es también el que produce mejor desempeño. La hipótesis, por tanto, es que las configuraciones con $\gamma \rightarrow 1$ deberían dominar el barrido; esta comprobación se presenta en la Sección 11.2.

Cada configuración fue entrenada de manera independiente y posteriormente evaluada mediante el número promedio de líneas eliminadas antes de alcanzar el estado terminal del juego.

La métrica de evaluación utilizada fue

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i, \quad (35)$$

donde L_i representa el número de líneas eliminadas en el episodio i y N corresponde al número total de episodios de evaluación.

La elección de esta métrica obedece a que el objetivo principal del agente consiste en maximizar el número de líneas eliminadas durante una partida de Tetris, lo que indirectamente prolonga la supervivencia.

11. Resultados Experimentales

Esta sección presenta los resultados obtenidos durante el barrido exhaustivo de hiperparámetros aplicado al agente de Aprendizaje por Refuerzo. El objetivo del experimento fue identificar la configuración que maximiza el número promedio de líneas eliminadas antes de alcanzar el estado terminal del juego. Todos los entrenamientos emplean la recompensa $R = L$ (líneas eliminadas, Sección 3).

En total se evaluaron veintisiete configuraciones independientes, obtenidas a partir de la combinación de tres valores para el factor de descuento γ , tres valores para la tasa de aprendizaje inicial α y tres valores para el número de episodios de decaimiento de la exploración N_ϵ .

La métrica principal de evaluación fue el promedio de líneas eliminadas por episodio:

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i, \quad (36)$$

donde L_i representa el número de líneas eliminadas en la partida i y N corresponde al número total de partidas consideradas en la evaluación.

11.1. Mejores configuraciones del barrido

La Tabla 1 muestra las diez configuraciones con mejor desempeño. Se observa que el mejor modelo corresponde a la configuración $\gamma = 0,999$, $\alpha = 0,0005$ y $N_\epsilon = 1500$, la cual alcanzó un promedio de 409,13 líneas eliminadas antes del estado terminal.

El mejor resultado cuantitativo del barrido fue:

Tabla 1: Diez mejores configuraciones del barrido.

Ranking	γ	α	N_ϵ	Líneas promedio
1	0.999	0.0005	1500	409.13
2	0.999	0.0005	1800	398.40
3	0.999	0.0001	1000	396.35
4	0.999	0.0001	1800	393.04
5	0.990	0.0010	1800	390.46
6	0.990	0.0001	1000	388.61
7	0.990	0.0005	1500	378.38
8	0.990	0.0005	1800	376.88
9	0.990	0.0001	1800	374.55
10	0.999	0.0001	1500	367.33

Tabla 2: Resumen del desempeño agrupado por factor de descuento.

γ	Promedio	Desviación estándar	Mínimo	Máximo
0.900	10.71	23.69	0.00	73.04
0.990	250.71	188.43	0.00	390.46
0.999	335.97	127.94	0.00	409.13

$$\gamma = 0,999, \quad \alpha = 0,0005, \quad N_\epsilon = 1500, \quad \bar{L} = 409,13. \quad (37)$$

Este resultado constituye la mejor política encontrada durante la experimentación y evidencia que el agente es capaz de aprender una estrategia de juego considerablemente superior a configuraciones menos favorables.

11.2. Resumen por factor de descuento

Esta subsección contrasta la hipótesis planteada en el diseño experimental (Sección 10): que un descuento interno cercano a 1, el más fiel al objetivo no descontado del problema, es también el que mejor desempeño produce. La Tabla 2 resume el desempeño agregado según el valor del factor de descuento.

Los resultados confirman la hipótesis. El desempeño promedio crece de forma monótona con γ (10,71 $\rightarrow$ 250,71 $\rightarrow$ 335,97): $\gamma = 0,999$ obtiene el mayor promedio, el mayor valor máximo (409,13) y una dispersión menor que la de $\gamma = 0,990$, por lo que es también el más robusto entre las configuraciones que aprenden. En contraste, $\gamma = 0,900$ prácticamente no aprende (promedio de 10,71 líneas): un horizonte demasiado corto impide valorar la supervivencia a largo plazo, esencial en Tetris, donde una colocación aparentemente conveniente en el corto plazo puede comprometer la partida. Se verifica así, empíricamente, que fijar el descuento interno de TD en un valor cercano a 1 es la mejor opción, en concordancia con el objetivo no descontado ($\gamma = 1$) definido en la Sección 3.6.

11.3. Resumen por tasa de aprendizaje

La Tabla 3 presenta el desempeño agrupado por tasa de aprendizaje inicial. Los promedios agregados muestran que el rendimiento depende de la interacción entre α , γ y el decaimiento de exploración, no solamente de un hiperparámetro aislado.

La tasa $\alpha = 0,0005$ obtiene el mejor promedio agregado (252,95) y el mejor resultado global, mientras que $\alpha = 0,001$ resulta la menos estable (promedio de 123,01). Esto sugiere que, con la recompensa $R = L$, una tasa de aprendizaje demasiado alta dificulta la convergencia de los pesos, y que un valor intermedio ofrece el mejor equilibrio entre velocidad y estabilidad.

Tabla 3: Resumen del desempeño agrupado por tasa de aprendizaje inicial.

α	Promedio	Desviación estándar	Mínimo	Máximo
0.0001	221.44	194.24	0.00	396.35
0.0005	252.95	188.38	0.00	409.13
0.0010	123.01	181.32	0.00	390.46

Tabla 4: Resumen del desempeño agrupado por episodios de decaimiento de exploración.

N_ϵ	Promedio	Desviación estándar	Mínimo	Máximo
1000	204.20	194.41	0.00	396.35
1500	129.01	192.27	0.00	409.13
1800	264.19	177.04	8.11	398.40

11.4. Resumen por decaimiento de exploración

La Tabla 4 muestra el desempeño agrupado por el número de episodios usados para reducir progresivamente la exploración. El mejor resultado puntual se obtuvo con $N_\epsilon = 1500$, mientras que el promedio agregado más alto corresponde a $N_\epsilon = 1800$. Esta diferencia indica que un decaimiento más prolongado mejora la robustez general (mayor mínimo y mayor promedio), pero la mejor configuración puntual se logra con un decaimiento intermedio.

11.5. Curva de aprendizaje del mejor modelo

La Figura 2 muestra la evolución del desempeño del mejor modelo durante el proceso de entrenamiento. Esta curva permite observar el comportamiento del agente conforme acumula experiencia y ajusta los pesos de la función de valor.

La curva de aprendizaje permite respaldar empíricamente que el agente no solo ejecuta una política fija, sino que modifica progresivamente su comportamiento a partir de la experiencia obtenida durante los episodios de entrenamiento.

11.6. Evolución de los pesos de la función de valor

La Figura 3 presenta la evolución temporal de los pesos asociados a la aproximación lineal de la función de valor. Cada peso representa la importancia asignada por el agente a una característica del tablero, como la altura agregada, los huecos, la rugosidad, las líneas eliminadas, la altura máxima o los pozos.

A diferencia de lo que cabría esperar de una convergencia clásica, las magnitudes de los pesos **no se estabilizan**: crecen de forma sostenida durante todo el entrenamiento (por ejemplo, el peso de la altura agregada alcanza valores del orden de -10^5). Este comportamiento es coherente con la inestabilidad conocida del aprendizaje TD semi-gradiente combinado con aproximación de funciones, y se ve acentuado por la agregación asíncrona de las actualizaciones entre procesos. La política, no obstante, sigue siendo competente porque depende únicamente de la *dirección* del vector de pesos y no de su magnitud: la selección por $\arg \max_a w^\top \varphi$ es invariante a escala, de modo que lo que se estabiliza son las *proporciones y los signos relativos* de los pesos, no sus valores absolutos. Esta divergencia de magnitudes motiva el uso de un método invariante a escala como el de entropía cruzada (Sección 7).

11.7. Comparación de agentes: TD frente a CEM

Para comparar de forma justa los dos métodos de entrenamiento se evaluaron ambos agentes (junto con los baselines aleatorio y heurístico) bajo un protocolo idéntico: 20 partidas con las mismas semillas, política *greedy* y un tope de 1000 colocaciones por partida. Por usar un tope

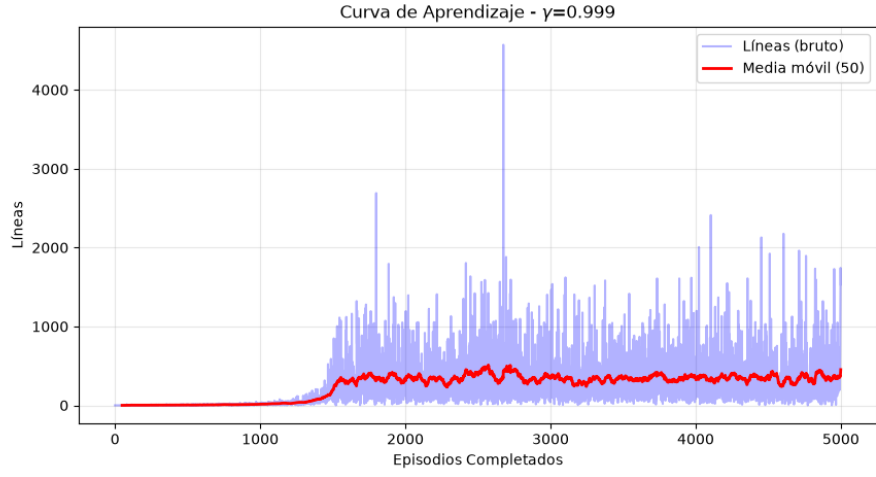


Figura 2: Curva de aprendizaje obtenida para la mejor configuración del barrido: $\gamma = 0,999$, $\alpha = 0,0005$ y $N_{\varepsilon} = 1500$.

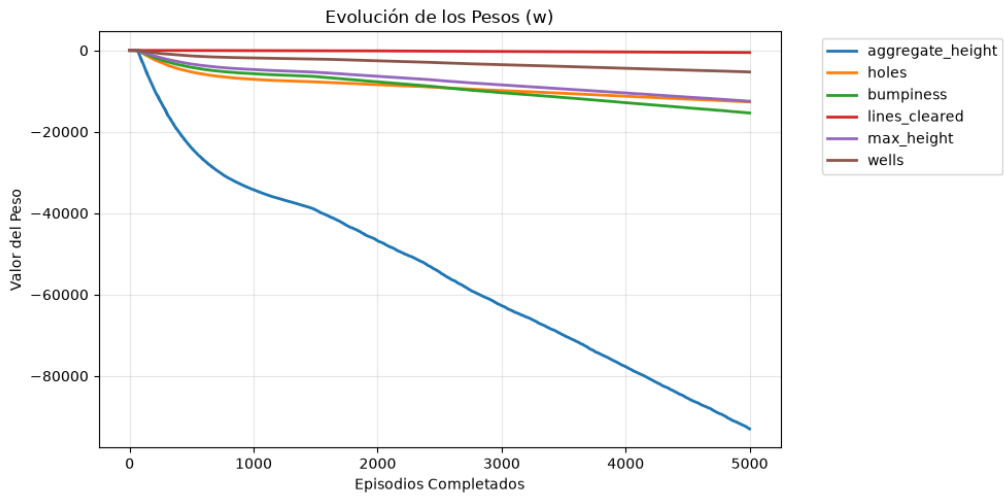


Figura 3: Evolución de los pesos de la función de valor para la mejor configuración de hiperparámetros.

Tabla 5: Comparación de agentes bajo un protocolo idéntico (20 partidas, tope de 1000 colocaciones, política greedy).

Agente	Líneas (media)	Desv. estándar	Máximo
Aleatorio	0.0	0.0	0
Heurístico	184.0	113.3	386
TD(0)	338.6	88.9	399
CEM	371.4	50.6	399

común, las cifras absolutas difieren del barrido sin tope de las secciones anteriores, pero permiten una comparación controlada. La Tabla 5 resume los resultados.

Bajo el mismo tope, CEM obtiene la mayor media de líneas (371,4 frente a 338,6 de TD) y, sobre todo, la menor desviación estándar (50,6 frente a 88,9), es decir, un desempeño más consistente. Ambos métodos aprenden políticas competentes y superan ampliamente al heurístico.

La diferencia más relevante está en la estabilidad de los pesos. Mientras que el vector de pesos de TD diverge en magnitud ($\|w\| \approx 10^5$), el de CEM permanece acotado y normalizado ($\|w\| = 1$). Los pesos de CEM son además directamente interpretables: penalizan la altura agregada, los huecos, la rugosidad y los pozos, y premian las líneas eliminadas, en concordancia con la intuición del juego. Este resultado confirma que un método de optimización directa e invariante a escala como CEM no solo evita la inestabilidad numérica del TD semi-gradiente, sino que produce una política igual o más efectiva.

11.8. Síntesis de resultados

El barrido experimental permitió identificar una configuración capaz de eliminar en promedio 409,13 líneas por partida antes de alcanzar el estado terminal. Este resultado confirma que la combinación de Aprendizaje por Refuerzo, aproximación lineal de funciones, representación mediante características y ajuste exhaustivo de hiperparámetros permite construir un agente competitivo para el entorno de Tetris considerado en este proyecto.

12. Discusión

Los resultados obtenidos permiten analizar el comportamiento del agente desde tres perspectivas: sensibilidad a los hiperparámetros, estabilidad del aprendizaje e interpretación de la política aprendida.

12.1. Sensibilidad a los hiperparámetros

El barrido exhaustivo muestra que el rendimiento del agente depende fuertemente de la combinación entre el factor de descuento, la tasa de aprendizaje y el calendario de exploración. La mejor configuración alcanzó un promedio de 409,13 líneas eliminadas, mientras que otras configuraciones obtuvieron desempeños considerablemente menores. Esta diferencia evidencia que el entrenamiento de agentes de Aprendizaje por Refuerzo no depende únicamente del algoritmo, sino también de una calibración adecuada de sus hiperparámetros.

El factor de descuento fue uno de los componentes más relevantes. Los valores $\gamma = 0,990$ y $\gamma = 0,999$ presentaron un desempeño promedio muy superior a $\gamma = 0,900$. Esto es coherente con el problema de Tetris, donde muchas decisiones correctas no producen una recompensa inmediata, sino que preparan el tablero para obtener mejores recompensas futuras. Por ejemplo, mantener una estructura baja y regular puede no eliminar líneas en el turno actual, pero aumenta la probabilidad de sobrevivir más tiempo y completar más líneas posteriormente.

La tasa de aprendizaje también mostró un efecto importante. El mejor resultado se obtuvo con $\alpha = 0,0005$, mientras que $\alpha = 0,001$ resultó la menos estable en promedio. Esto indica que una

tasa demasiado alta acelera la adaptación de los pesos pero aumenta el riesgo de inestabilidad, y que un valor intermedio ofrece el mejor equilibrio entre velocidad y estabilidad. Por ello, el mejor resultado surge de una interacción favorable entre $\alpha = 0,0005$, $\gamma = 0,999$ y $N_\varepsilon = 1500$.

12.2. Exploración y explotación

El parámetro N_ε controla la rapidez con la que disminuye la exploración del agente. En la mejor configuración, el valor $N_\varepsilon = 1500$ ofreció un equilibrio adecuado entre exploración y explotación. Esto sugiere que, para el entorno estudiado, el agente requiere una fase de exploración moderada para cubrir el espacio de estados antes de consolidar las acciones de mayor valor estimado.

Sin embargo, el análisis agregado muestra que $N_\varepsilon = 1800$ obtuvo el promedio más alto entre los tres valores evaluados. Esto indica que una exploración más prolongada puede mejorar la robustez general del entrenamiento, aunque no necesariamente produce la mejor configuración individual. Por tanto, existe un compromiso entre explorar durante más tiempo para cubrir mejor el espacio de estados y explotar más temprano para estabilizar la política.

12.3. Interpretación de la curva de aprendizaje

La Figura 2 permite observar la evolución del agente durante el entrenamiento. En este tipo de problemas es esperable que la curva no sea estrictamente monótona, debido a la aleatoriedad en la generación de piezas y a la exploración inducida por la política ε -greedy.

Aun con dicha variabilidad, la curva resulta útil para verificar que el agente experimenta una mejora progresiva y que el entrenamiento no corresponde a una ejecución aleatoria sin aprendizaje. La existencia de una tendencia de mejora y posterior estabilización respalda que la función de valor aproxima gradualmente una valoración útil de los estados del tablero.

12.4. Interpretación de la evolución de pesos

La Figura 3 permite analizar cómo cambian los parámetros de la función de valor durante el entrenamiento. En una aproximación lineal, cada peso determina la influencia de una característica sobre la evaluación del tablero. Por ello, la evolución de los pesos proporciona una forma de interpretación del comportamiento aprendido por el agente.

Las magnitudes de los pesos no se estabilizan, sino que crecen de forma sostenida durante el entrenamiento, fenómeno propio del aprendizaje TD semi-gradiente con aproximación de funciones. No obstante, lo que sí se estabiliza son las proporciones y los signos relativos de los pesos: el agente aprende a penalizar configuraciones desfavorables, como tableros altos, huecos internos o superficies irregulares, y a favorecer las que incrementan la supervivencia y la eliminación de líneas. Como la política depende únicamente de la dirección del vector de pesos (invarianza de escala), sigue siendo efectiva pese a la divergencia de magnitud; esta limitación motiva el uso del Método de Entropía Cruzada (Sección 7).

12.5. Alcance del resultado obtenido

El mejor modelo logró eliminar en promedio 409,13 líneas antes del estado terminal. Este resultado es relevante porque fue obtenido mediante una aproximación lineal relativamente simple, sin redes neuronales profundas ni búsqueda exhaustiva de largo horizonte. Por tanto, el proyecto demuestra que una representación adecuada del estado mediante características puede ser suficiente para construir un agente efectivo en una versión simplificada del problema de Tetris.

No obstante, el resultado debe interpretarse dentro de las condiciones del entorno implementado. El agente decide por colocaciones completas y no modela maniobras avanzadas dependientes

del tiempo real, como desplazamientos durante la caída, rotaciones tardías o técnicas especializadas. En consecuencia, el desempeño obtenido corresponde al modelo de decisión definido en el MDP del proyecto y no necesariamente a todas las variantes posibles del juego Tetris.

12.6. Comparación con trabajos previos

El desempeño obtenido, de aproximadamente 409 líneas por partida, se encuentra por debajo de los mejores agentes reportados en la literatura, algunos de los cuales superan decenas de miles de líneas utilizando representaciones más complejas y algoritmos de optimización avanzados. Sin embargo, el resultado obtenido es significativo considerando que el agente utiliza una aproximación lineal y un conjunto reducido de características.

12.7. Limitaciones

La principal limitación del enfoque es el uso de una función de valor lineal. Aunque esta elección mejora la interpretabilidad y reduce el costo computacional, también limita la capacidad del agente para capturar relaciones no lineales complejas entre las características del tablero. Además, el entrenamiento sigue siendo sensible a la elección de hiperparámetros, como se evidenció en el barrido experimental.

Otra limitación es que la recompensa se basa principalmente en líneas eliminadas. Aunque esta métrica está alineada con el objetivo del juego, puede no capturar completamente criterios estratégicos intermedios, como preparar el tablero para futuras piezas, reducir riesgos de top-out o mantener opciones abiertas para distintas secuencias de piezas.

12.8. Implicancias para trabajos futuros

Los resultados obtenidos abren la posibilidad de extender el sistema hacia enfoques más expresivos, como Deep Q-Learning, redes neuronales convolucionales para representar directamente el tablero o métodos híbridos que combinen heurísticas con Aprendizaje por Refuerzo. También sería posible ampliar el espacio de acciones para incluir maniobras más cercanas al juego real, así como comparar el agente entrenado contra políticas aleatorias, heurísticas clásicas y controladores basados en búsqueda.

En síntesis, los experimentos confirman que la aproximación lineal de la función de valor es una solución viable y efectiva para el entorno propuesto, tanto con TD(0) como, de forma más robusta, con el Método de Entropía Cruzada. La combinación de resultados cuantitativos, curva de aprendizaje y evolución de pesos proporciona evidencia suficiente para sostener que el agente aprende una política útil y no se limita a una estrategia aleatoria o fija.

13. Conclusiones

El presente trabajo abordó el problema de la toma de decisiones en el juego Tetris mediante técnicas de Aprendizaje por Refuerzo y optimización estocástica. El entorno fue modelado formalmente como un Proceso de Decisión de Markov, permitiendo representar el problema dentro del marco teórico del Aprendizaje por Refuerzo.

Con el fin de reducir la complejidad del espacio de estados, se empleó una representación basada en características geométricas del tablero, entre las que destacan la altura agregada, el número de huecos, la rugosidad de la superficie, las líneas eliminadas, la altura máxima y los pozos.

La función de valor fue aproximada mediante un modelo lineal cuyos parámetros se optimizaron utilizando dos estrategias diferentes: Aprendizaje por Diferencias Temporales TD(0) y el Método de Entropía Cruzada.

Los experimentos realizados mediante un barrido exhaustivo de hiperparámetros permitieron identificar una configuración de entrenamiento capaz de eliminar en promedio

$$\bar{L} = 409,13 \quad (38)$$

líneas antes de alcanzar el estado terminal, evidenciando la capacidad del agente para aprender políticas de juego competitivas.

Más allá del barrido, la comparación controlada entre ambos métodos de entrenamiento mostró que el Método de Entropía Cruzada iguala o supera a TD(0) en desempeño (371,4 frente a 338,6 líneas promedio, con menor variabilidad) y, además, mantiene el vector de pesos acotado e interpretable ($\|w\| = 1$). En contraste, los pesos de TD(0) divergen en magnitud, una manifestación de la inestabilidad del aprendizaje semi-gradiente con aproximación de funciones, aunque su política sigue siendo efectiva por la invarianza de escala. Esto posiciona al Método de Entropía Cruzada como la alternativa más robusta para el problema considerado.

Los resultados obtenidos muestran que las técnicas de aproximación lineal de funciones constituyen una alternativa efectiva para abordar problemas secuenciales de gran complejidad sin necesidad de recurrir a modelos de Aprendizaje Profundo o arquitecturas neuronales de gran escala.

Finalmente, la naturaleza modular de la arquitectura desarrollada permite la incorporación de nuevos algoritmos de aprendizaje, convirtiendo al sistema propuesto en una plataforma adecuada para futuras investigaciones en Aprendizaje por Refuerzo aplicado a videojuegos y problemas de optimización secuencial.

Referencias

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Bertsekas, D. P. and Tsitsiklis, J. N. (1996). *Neuro-Dynamic Programming*. Athena Scientific.
- Dellacherie, P. (2003). Tetris artificial intelligence. *France Telecom Research*.
- Gabillon, V., Ghavamzadeh, M., and Scherrer, B. (2013). Approximate dynamic programming finally performs well in the game of Tetris. In *Advances in Neural Information Processing Systems (NIPS)*, volume 26, pages 1754–1762.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- Rubinstein, R. Y. (1999). The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190.
- Russell, S. and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.
- Szita, I. and Lőrincz, A. (2006). Learning Tetris using the noisy cross-entropy method. *Neural Computation*, 18(12):2936–2941.
- Thiery, C. and Scherrer, B. (2009). Building controllers for Tetris. *ICGA Journal*, 32(1):3–11.